

Universidad de Alcalá

Escuela Politécnica Superior
Calidad y Pruebas del Software

Práctica 2

Calidad, Fiabilidad y Mantenimiento

Daniel Godoy Velles

Nacho Apellido1 Apellido2

Jaime Apellido1 Apellido2

16 de abril de 2026

Índice

1. Parte 1: Herramientas de medida de software	2
1.1. Análisis de métricas (Radon)	2
1.2. Interpretación y Justificación	3
2. Parte 2: Fiabilidad del sistema	5
2.1. Implementación de la herramienta de cálculo	5
2.2. Indicadores de Fiabilidad	5
2.3. Representación Gráfica y Curva de Fiabilidad	5
3. Parte 3: Mantenimiento y mejora de calidad	6
3.1. Tests desarrollados	6
3.2. Análisis de mejoras propuestas	9
3.3. Cambios realizados en el código	9
3.4. Comparativa de resultados	10

1. Parte 1: Herramientas de medida de software

En esta sección se analiza el proyecto utilizando la herramienta Radon para extraer métricas de software relevantes.

1.1. Análisis de métricas (Radon)

Fichero	Tipo	Línea	Nombre	CC	Grado
texttest_fixture.py	F	7:0	main	4	A
gilded_rose.py	M	8:4	GildedRose.update_quality	19	C
gilded_rose.py	C	3:0	GildedRose	11	C
gilded_rose.py	C	39:0	Item	2	A
gilded_rose.py	M	5:4	GildedRose.__init__	1	A
gilded_rose.py	M	40:4	Item.__init__	1	A
gilded_rose.py	M	45:4	Item.__repr__	1	A
src/python/__init__.py	F	1:0	hello	1	A
tests/test_gilded_rose_approvals.py	F	7:0	test_gilded_rose_approvals	1	A

Cuadro 1: Resultados de Complejidad Ciclomática (Radon CC)

Fichero	LOC	LLOC	SLOC	Coment.	Blank
texttest_fixture.py	36	22	29	2	6
gilded_rose.py	46	39	39	1	6
src/python/__init__.py	2	2	2	0	0
tests/test_gilded_rose_approvals.py	21	17	17	0	4
tests/test_gilded_rose.py	16	10	10	1	5
tests/__init__.py	0	0	0	0	0
tests/conftest.py	0	0	0	0	0

Cuadro 2: Métricas Raw (Radon RAW)

Fichero	n_1	n_2	N_1	N_2	Vocab.	Vol.	Diff.	Effort	Bugs
texttest_fixture.py	5	12	9	17	17	106.27	3.54	376.39	0.035
gilded_rose.py	8	15	27	54	23	366.41	14.40	5276.28	0.122
tests/.../approvals.py	1	2	1	2	3	4.75	0.50	2.38	0.002
tests/test_gilded_rose.py	1	2	1	2	3	4.75	0.50	2.38	0.002
src/python/__init__.py	0	0	0	0	0	0.00	0.00	0.00	0.000
tests/__init__.py	0	0	0	0	0	0.00	0.00	0.00	0.000
tests/conftest.py	0	0	0	0	0	0.00	0.00	0.00	0.000

Cuadro 3: Métricas de Halstead (Radon HAL)

Fichero	Score	Grado
texttest_fixture.py	70.99	A
gilded_rose.py	53.77	A
src/python/__init__.py	100.00	A
tests/test_gilded_rose_approvals.py	68.15	A
tests/test_gilded_rose.py	61.88	A
tests/__init__.py	100.00	A
tests/conftest.py	100.00	A

Cuadro 4: Maintainability Index (Radon MI)

1.2. Interpretación y Justificación

Antes de proceder con la comparativa, es fundamental entender qué mide cada una de las métricas extraídas:

- **Complejidad Ciclomática (CC):** Mide el número de caminos linealmente independientes a través del código, el cual aumenta con el número de bifurcaciones. Una complejidad alta indica una lógica de control difícil de seguir y testear. Por ejemplo, el número de casos de prueba necesario para cubrir completamente el código aumenta exponencialmente con la complejidad.
- **Métricas Raw (RAW):** Proporcionan conteos básicos que permiten evaluar el tamaño físico del software y cuán documentado está el código: **LOC** (*Lines of Code*) cuenta el total de líneas del fichero; **LLOC** (*Logical Lines of Code*) cuenta las instrucciones lógicas (asignaciones, condicionales, llamadas, etc.); **SLOC** (*Source Lines of Code*) cuenta únicamente las líneas con código real, excluyendo comentarios y líneas en blanco, siendo el indicador más representativo del tamaño efectivo; **Coment.** recoge las líneas de comentario, cuya proporción respecto a SLOC es un indicador directo de documentación y, por ende, de mantenibilidad; y **Blank** contabiliza las líneas en blanco, que influyen en la legibilidad.
- **Métricas de Halstead (HAL):** Evalúan la complejidad algorítmica basándose en el número de operadores y operandos únicos usados (n_1, n_2) y el número de operadores y operandos totales sin unicidad (N_1, N_2). Incluye el *Vocabulario* (número de operadores y operandos únicos), el *Volumen* (tamaño de la implementación) y el *Esfuerzo* (carga mental para entender el código). Además, estima la probabilidad de errores (Bugs) basándose en el volumen y el vocabulario. Un esfuerzo alto indica que el código es difícil de entender y mantener, lo que puede traducirse en una mayor probabilidad de introducir errores en futuras modificaciones.
- **Índice de Mantenibilidad (MI):** Un valor compuesto que estima la facilidad con la que el software puede ser mantenido, combinando CC, Halstead y el tamaño del código. A mayor valor, mejor mantenibilidad. Sin embargo, es importante destacar que el MI puede ser engañoso en archivos pequeños, ya que tiende a ser artificialmente alto en este caso.

Al realizar un análisis comparativo entre los distintos módulos del proyecto, se observa una clara disparidad en la calidad del código, concentrándose la mayor deuda técnica en el archivo `gilded_rose.py`.

- **Comparativa de Complejidad (CC):** Mientras que los módulos de apoyo como `texttest_fixture.py` y los tests mantienen una complejidad de grado **A** (valores entre 1 y 5), el archivo `gilded_rose.py` destaca negativamente con un grado **C** (valores entre 11 y 20) en su método principal. Esta diferencia subraya que en el archivo `gilded_rose.py`, en específico en el método `update_quality`, la lógica de control es mucho más intrincada debido a un mayor número de bifurcaciones, lo que dificulta su comprensión y mantenimiento.
- **Distribución del Esfuerzo (Halstead):** El contraste en el esfuerzo mental (Effort) es el más revelador. El archivo `gilded_rose.py` registra un valor de **5276.28**, lo que supone casi 14 veces más esfuerzo que el siguiente archivo más complejo (`texttest_fixture.py` con 376.39) y miles de veces más que los archivos de test. Esto posiciona a `gilded_rose.py` como el cuello de botella crítico para cualquier mantenimiento futuro.
- **Densidad de Errores y Tamaño:** A pesar de que todos los archivos tienen un tamaño similar en líneas de código (RAW), con `gilded_rose.py` (39 SLOC) y `texttest_fixture.py` (29 SLOC) muy parejos, la probabilidad de bugs estimada por Halstead es notablemente superior en el primero (**0.122** frente a **0.035**). Esto confirma que no es el tamaño del archivo lo que genera el riesgo, sino la densidad de operaciones y operandos.
- **Limitaciones del Índice de Mantenibilidad (MI):** Curiosamente, todos los archivos presentan un grado **A** en el índice MI. Sin embargo, al compararlo con el esfuerzo de Halstead, se hace evidente que el MI tiende a ser optimista en archivos pequeños, ocultando la complejidad real que sí detectan las métricas de CC y Effort.

2. Parte 2: Fiabilidad del sistema

Análisis del comportamiento de la aplicación desde el punto de vista de la fiabilidad, simulando un entorno de servidor propenso a fallos.

2.1. Implementación de la herramienta de cálculo

Resumen de cómo se ha implementado el archivo Excel para el cálculo automático de indicadores básico de fiabilidad.

2.2. Indicadores de Fiabilidad

Documentar los indicadores calculados: número de fallos, MTBF, MTTF, MTTR, disponibilidad, etc.

2.3. Representación Gráfica y Curva de Fiabilidad

Incluir gráficas desarrolladas de los periodos de funcionamiento y fallo, así como la curva de fiabilidad $R(t) = e^{-\lambda \cdot t}$.

3. Parte 3: Mantenimiento y mejora de calidad

Propuesta y ejecución de mejoras en el código para optimizar el diseño y mejorar las métricas obtenidas anteriormente.

Lo primero que se ha realizado es un conjunto de tests a partir de la funcionalidad indicada en los requisitos de Gilded Rose, para posteriormente realizar una refactorización del código y mejorar las métricas obtenidas en la Parte 1. De esta manera, podemos asegurar el correcto funcionamiento del código mientras es refactorizado.

3.1. Tests desarrollados

A continuación se muestran los tests desarrollados para cada uno de los tipos de ítems del sistema, así como los tests de invariantes globales (calidad nunca negativa ni superior a 50):

```
def test_non_specified_item():
    """
    Tests that normal items quality decays 1 each day and 2 after sell_in <= 0
    """
    gilded_rose = GildedRose([Item("+5 Dexterity Vest", 10, 20)])
    item = Item("+5 Dexterity Vest", 10, 20)

    for _ in range(20):
        # calidad baja 1 (o 2 si sell_in <= 0). Nunca baja de 0.
        item.quality = item.quality - (1 if item.sell_in > 0 else 2)
        item.quality = max(0, item.quality)

        # sell_in siempre baja 1
        item.sell_in = item.sell_in - 1

        # logica real de la clase
        gilded_rose.update_quality()

    expected_values = (item.quality, item.sell_in)

    actual_values = (gilded_rose.items[0].quality, gilded_rose.items[0].sell_in)

    assert actual_values == expected_values
```

Figura 1: Test: ítems sin categoría especial

```
def test_sulfuras_hand_of_ragnaros():
    sulfuras = Item("Sulfuras, Hand of Ragnaros", 0, 80)
    gilded_rose = GildedRose([sulfuras])
    for _ in range(20):
        gilded_rose.update_quality()

    assert sulfuras.sell_in == 0
    assert sulfuras.quality == 80
```

Figura 2: Test: Sulfuras, Hand of Ragnaros

```
def tests_aged_brie_quality_increases():
    aged_brie = Item("Aged Brie", 5, 7)
    gilded_rose = GildedRose([Item("Aged Brie", 5, 7)])

    for _ in range(20):
        # manual update
        aged_brie.quality += 1 if aged_brie.sell_in > 0 else 2
        aged_brie.sell_in -= 1

        # actual logic
        gilded_rose.update_quality()

    aged_brie.quality = min(50, aged_brie.quality) # nunca mayor de 50

    expected_values = (aged_brie.quality, aged_brie.sell_in)
    actual_values = (gilded_rose.items[0].quality, gilded_rose.items[0].sell_in)

    assert actual_values == expected_values
```

Figura 3: Test: Aged Brie

```
def tests_backstage_passes_increases_quality_then_drops():
    backstage_passes = Item("Backstage passes to a TAFKAL80ETC concert", 5, 7)
    gilded_rose = GildedRose([Item("Backstage passes to a TAFKAL80ETC concert", 5, 7)])

    for _ in range(20):
        # manual update
        backstage_passes.sell_in -= 1
        if backstage_passes.sell_in > 0:
            backstage_passes.quality += 1
        else:
            backstage_passes.quality = 0

        # actual logic
        gilded_rose.update_quality()

    expected_values = (backstage_passes.quality, backstage_passes.sell_in)
    actual_values = (gilded_rose.items[0].quality, gilded_rose.items[0].sell_in)

    assert actual_values == expected_values
```

Figura 4: Test: Backstage passes


```

def tests_conjured_items_degrade_twice_as_fast():
    conjured = Item("Conjured Mana Cake", 3, 6)
    gilded_rose = GildedRose([Item("Conjured Mana Cake", 3, 6)])

    for _ in range(20):
        # manual update
        conjured.quality = conjured.quality - (1 if conjured.sell_in > 0 else 2)
        conjured.quality = max(0, conjured.quality)

        # sell_in siempre baja 1
        conjured.sell_in = conjured.sell_in - 1

        # actual logic
        gilded_rose.update_quality()

    expected_values = (conjured.quality, conjured.sell_in)
    actual_values = (gilded_rose.items[0].quality, gilded_rose.items[0].sell_in)

    assert actual_values == expected_values

```

Figura 5: Test: Conjured Mana Cake

```

def tests_no_item_quality_is_negative():
    gilded_rose = GildedRose(
        [
            Item(name="+5 Dexterity Vest", sell_in=10, quality=20),
            Item(name="Aged Brie", sell_in=2, quality=0),
            Item(name="Elixir of the Mongoose", sell_in=5, quality=7),
            Item(name="Sulfuras, Hand of Ragnaros", sell_in=0, quality=80),
            Item(name="Sulfuras, Hand of Ragnaros", sell_in=-1, quality=80),
            Item(name="Backstage passes to a TAFKAL80ETC concert", sell_in=15, quality=20),
            Item(name="Backstage passes to a TAFKAL80ETC concert", sell_in=10, quality=49),
            Item(name="Backstage passes to a TAFKAL80ETC concert", sell_in=5, quality=49),
            Item(name="Conjured Mana Cake", sell_in=3, quality=6),
        ]
    )

    for _ in range(100):
        gilded_rose.update_quality()

    for item in gilded_rose.items:
        assert item.quality >= 0

```

Figura 6: Test: la calidad nunca es negativa

```
def tests_no_item_quality_is_greater_than_50(): # except sulfuras
    gilded_rose = GildedRose(
        [
            Item(name="+5 Dexterity Vest", sell_in=10, quality=1000),
            Item(name="Aged Brie", sell_in=2, quality=100),
            Item(name="Elixir of the Mongoose", sell_in=5, quality=200),
            Item(name="Backstage passes to a TAFKAL80ETC concert", sell_in=15, quality=20),
            Item(name="Backstage passes to a TAFKAL80ETC concert", sell_in=10, quality=60),
            Item(name="Backstage passes to a TAFKAL80ETC concert", sell_in=5, quality=49),
            Item(name="Conjured Mana Cake", sell_in=3, quality=6),
        ]
    )

    for _ in range(50):
        gilded_rose.update_quality()

    for item in gilded_rose.items:
        assert item.quality <= 50
```

Figura 7: Test: la calidad nunca supera 50

3.2. Análisis de mejoras propuestas

A partir de los tests desarrollados y del análisis de métricas de la Parte 1, se identifican las siguientes mejoras:

- **Separación de lógica por tipo de ítem:** Analizar y encapsular la lógica para cada tipo de objeto (Aged Brie, Sulfuras, Backstage Passes y cualquier otro) de forma independiente, facilitando la lectura y la comprensión de cada caso.
- **Sustitución de bifurcaciones por operaciones matemáticas:** Reemplazar la lógica de limitación de calidad ($0 \leq \text{calidad} \leq 50$) por el uso de `max` y `min`, reduciendo el número de ramas y, por tanto, la complejidad ciclomática.
- **Generalización del flujo común:** Introducir en el flujo de control general las modificaciones que se aplican a todos los objetos (o a la mayoría), dejando las excepciones como casos particulares.
- **Documentación del código:** Añadir comentarios explicativos en cada bloque lógico para facilitar su comprensión y, por ende, su mantenibilidad futura.
- **Eliminación de código duplicado:** Simplificar la lógica de cada tipo de ítem suprimiendo redundancias y mejorando la claridad del flujo.

3.3. Cambios realizados en el código

Aplicando las mejoras anteriores, el código refactorizado resultante es el siguiente:

```

def update_quality(self):
    """
    Actualiza el estado de cada uno de los items.
    """
    for item in self.items:

        if item.name == "Sulfuras, Hand of Ragnaros":
            # no se modifica nada
            continue

        if item.name == "Aged Brie":
            # gana valor con el tiempo
            item.quality += 1

            # el doble de valor cuando ha pasado la fecha de venta
            if item.sell_in <= 0:
                item.quality += 1

        elif item.name == "Backstage passes to a TAFKAL80ETC concert":
            # cuando ha pasado la fecha, no tiene valor
            if item.sell_in <= 0:
                item.quality = 0
            # cuando se acerca mucho sube + 3
            elif item.sell_in <= 5:
                item.quality += 3
            # cuando se acerca bastante sube +2
            elif item.sell_in <= 10:
                item.quality += 2

        else:
            # items normales se degradan con el tiempo
            item.quality -= 1

            # los items se degradan el doble cuando se ha pasado la fecha de venta
            if item.sell_in <= 0:
                item.quality -= 1

    # un día menos
    item.sell_in -= 1
    item.quality = min(self.max_quality, max(self.min_quality, item.quality))

```

Figura 8: Código refactorizado de `gilded_rose.py`

No obstante, estas mejoras se realizan respetando las restricciones indicadas en los requisitos originales. Si se quisiera una mayor mantenibilidad y seguir más estrictamente el principio de responsabilidad única, se podría definir un conjunto de comportamientos asignados a los objetos mediante composición. En otras palabras, se podría aplicar el *Strategy Pattern* para encapsular el comportamiento de cada tipo de ítem de forma intercambiable.

3.4. Comparativa de resultados

Las siguientes tablas comparan las métricas de `gilded_rose.py` antes y después de la refactorización.

Nombre	Tipo	Grado anterior	Grado actual	Mejora
GildedRose.update_quality	M	C (19)	B (10)	↑
GildedRose	C	C (11)	B (7)	↑

Cuadro 5: Comparativa de Complejidad Ciclomática (Radon CC)

Métrica	LOC	LLOC	SLOC	Coment.	Blank
Antes	46	39	39	1	6
Después	76	38	33	9	13

Cuadro 6: Comparativa de Métricas Raw (Radon RAW)

Estado	n_1	n_2	N_1	N_2	Vocab.	Vol.	Diff.	Effort	Bugs
Antes	8	15	27	54	23	366.41	14.40	5276.28	0.122
Después	5	14	16	32	19	203.90	5.71	1165.15	0.068

Cuadro 7: Comparativa de Métricas de Halstead (Radon HAL)

Estado	Score	Grado
Antes	53.77	A
Después	74.36	A

Cuadro 8: Comparativa de Maintainability Index (Radon MI)

La refactorización produce mejoras en todas las métricas relevantes. En CC, tanto `update_quality` como la clase `GildedRose` reducen su grado de **C** a **B**, gracias a la separación de la lógica en bloques independientes por tipo de ítem. En Halstead, el esfuerzo cae de **5276.28** a **1165.15** (una reducción del 78 %), y la probabilidad de bugs estimada mejora de **0.122** a **0.068**. En Raw, el SLOC efectivo se reduce de 39 a 33 líneas, mientras que los comentarios pasan de 1 a 9, reflejando la mayor documentación introducida. El MI mejora de 53.77 a 74.36, manteniéndose en grado **A** y confirmando que la refactorización ha mejorado la mantenibilidad global.